Re-proving `native_decide` facts in Lean's kernel: a study of 16 public packages and a converter

Tejasvi Singh Tomar

October 2026

Abstract

Lean's `native_decide` proves a decidable proposition by running compiled code and adds an axiom that records the result. Mathlib refuses such axioms, and so do the gates of Lean 4.35 (`lake check` and Comparator) and the independent proof checkers. We present NativeTrust, a Lean library that re-proves the recorded facts by kernel evaluation without editing the projects that contain them, and names the blocking constant when it finds one. Across the 52,821 native facts in 16 public packages (counting only built modules whose source names a native tactic), nine packages converted at least 90% of their facts, and the other seven converted at most 39%. The pooled rate is 98.0%, but one package contributes 87% of the facts, and the rate without the three largest packages is 65.9%. With all of the converter's transformations switched off, plain kernel evaluation alone reproduced 51,307 of 51,329 re-run conversions, so nearly every fact that converted never needed compiled evaluation. Of the other 22, six need their checks split into smaller declarations, five of them only because a static check caps the unsplit evaluation. The remaining 16 evaluate a definition compiled through `WellFounded.fix`. For these we give a fuel-indexed replacement, proved from the unfolding equation, that reduces no accessibility proofs, and it converts the two functions we tested on a nightly that contains core PR #14892. A one-import switch, which tries `decide +kernel` first, removed all 1,113 native axioms from lido-srv3 and from cvc5/logos's ten largest modules, with build time changed by at most 2%. It removed 36% of the native axioms of a random sample of 24 packages at 1.3 times the build time, and 21 of the 24 still built. A source rewriter replaces `native_decide` by `decide +kernel` wherever the kernel proves the facts within a per-fact budget. It removed 30% of the sample's native axioms (25% in the median package) and kept every build within 1.7 times its original build time. Without the budget it removed 54% (76% in the median package) but made one build 143.5 times slower. In both settings 23 of the 24 packages still built. Of the study's 1,057 failures, 825 are traced to a construct that the kernel cannot evaluate, 585 of them to iterator-based string functions. All but two of the 232 untraced failures exceed our time, memory or recursion-depth limits, mostly in the converter's own search for split points. Extensions made after the study address these causes: linear evaluation of string literals and proved replacements for string functions; cast-free decidability instances; rewriting of definitions that occur in types; budgets, structured checks and sharding for large checks; and a kernel-checked route for `bv_decide` with a bit-blaster written for the kernel. That route converts all 213 `bv_decide` facts of the study and the sample. BerryEsseen, the source of 569 of the 585 string-function failures, now converts all 579 of its facts. Some of the extensions were developed against the random sample's failures, and they raised its pooled rate from 44.9% to 68.8% and its per-package median from 77.8% to 100%. We produced kernel supplements for three packages, 1,594 theorems in all: one for every theorem whose native facts all converted, in the modules counted above. Each states the same proposition as its original, or a definitionally equal one, and uses only the standard axioms. All of them passed nanoda, while con-ron accepted 246 of the 248 supplement modules. The hand-built kernel versions of our own published results took about 1.03 million CPU-hours to build. The converter replaced two of them at about their cost and gave kernel proofs of 543 of

the 770 core theorems of a result that had none, but no headline theorem converted in the 16 other results that we ran, where most facts exceeded our time, memory or recursion-depth limits.

1 Introduction

The tactic `native_decide` closes a goal p by compiling the decision procedure `decide` p and running it. Since Lean 4.29 each use adds an axiom `<decl>._native.native_decide.ax_N : decide p = true`; the tactics `decide +native` and `bv_decide` add similar axioms of the form $e =$ `true`. A proof that uses one trusts Lean's compiler and runtime and every `@[implemented_by]` and `@[extern]` implementation that the computation reaches. Mathlib forbids the tactic. Both `lake check` and Comparator admit only `propext`, `Classical.choice` and `Quot.sound`, and the independent checkers cannot check compiled evaluation at all. Even so, in October 2026, GitHub code search listed 585 repositories whose Lean files mention the tactic, 575 of which received pushes in 2026. Of the 30 we examined, 16 contain native facts that build.

One remedy is to rewrite the decision procedures by hand so that the kernel can evaluate them, and to build them in shards. Our own projects show its cost. A fresh build of the kernel-checked release of the sorting-network bound $S(13) \geq 45$ took about 8,800 CPU-hours according to the release's build log [31]. For 15 theorems of a quantum error-correction paper [32], the kernel-checked version needed 39 hand-written files and four generators of check modules, and building its 4,777 kernel modules took 715.6 CPU-hours in a clean build. By their own records, the hand-built kernel versions of 23 of our published results took about 1.03 million CPU-hours to build. We measure how many native facts can instead be converted automatically, classify the rest, and compare the two routes on our own results (Section 8).

Contributions.

- A study of the 52,821 native facts in 16 public packages, with an ablation showing that plain kernel evaluation suffices for all but 22 of 51,329 re-run conversions, a classification of the 1,057 failures (825 traced to a construct), a re-run of 56 blocked facts without the evaluation cap, and a random sample of repositories (Section 4).
- A one-import switch that makes every `native_decide` try `decide +kernel` first, a linter that flags unnecessary uses with an automatic fix, and a source rewriter whose per-fact budget kept every build of a random sample of packages within 1.7 times its original build time (Section 5).
- NativeTrust, a converter that leaves the source project unchanged and writes a kernel supplement for it (Section 3). It handles facts inside definitions and instances, accepts externally proved facts and proved replacements, bounds evaluations that cannot finish, and replaces well-founded recursion by a fuel-indexed version that reduces no proofs, with a recorded fallback to accessibility proofs.
- Independent checks: kernel supplements for three packages, checked for statement identity and axioms by a separate script and by nanoda and con-ron, and samples from the other packages (Section 6).
- Extensions after the study that close most of the blocked classes, each gated by re-running about 7,300 previously converted facts (Section 4.8); two remedies, one for primality inside instances and one for string functions (Section 4.4); and a kernel-checked route for `bv_decide` (Section 7): a bit-blaster written for the kernel and an LRAT checker on natural-number

bitmasks, both proved sound. The route converts all 213 `bv_decide` facts of the corpus and the random sample; the largest study fact (65,720 clauses) takes 247 s of kernel time. A drop-in tactic on the same route converts 190 of the 208 `bv_decide` axioms that the rewriter leaves in the random sample.

- Runs on our own published results (Section 8). The converter replaces the hand-built kernel versions of a quantum error-correction theorem and of a fine-grained complexity result at about their cost, and gives kernel proofs of 543 of the 770 core theorems of a result that had no kernel version; in 16 other results, among them the sorting-network bound, no headline theorem converts.

2 Background

Lean's kernel evaluates closed terms by reduction, with built-in arithmetic on natural-number literals; `decide +kernel` uses it. Four things stop kernel evaluation.

1. *Well-founded recursion.* Lean 4.27–4.36 compile a definition whose termination measure is not a natural number to `WellFounded.fix (Lean.opaqueId wf) F`. The kernel cannot unfold `opaqueId`, so it cannot evaluate, for example, `List.mergeSort`. Natural-number measures compile to the fuel-based `WellFounded.Nat.fix`, which reduces.

2. *Constants without a usable definition.* These include `partial` functions (opaque in the logic), C primitives declared `@[extern]`, the platform word size behind `USize` and `HashMap`, and definitions that use `Classical.choice` computationally, such as `partial_fixpoint` definitions (`Lean.Order.fix`).

3. *Casts along propositional equalities.* `Eq.rec` along an equality proved with `propext` or `Quot.sound` does not reduce unless its two sides are definitionally equal; compiled code erases such casts.

4. *Cost.* The kernel interprets terms by reduction, which is slower than running compiled code; within one declaration its caches grow with the length of the evaluation, and deep recursion can exhaust its stack.

3 Method

Definitions. A fact is *converted* when NativeTrust produces a theorem with the fact's statement (or its restatement over rebuilt definitions, Section 3.1) that Lean's kernel accepts within the study's time and memory limits (Section 4) and that depends only on the three standard axioms. A *blocking constant* is a constant at which kernel evaluation stops when it is used for computation: an axiom (including `Classical.choice`, which the logic admits but the kernel cannot compute with), an opaque constant (including `partial` and `@[extern]` functions without a definition), or a definition that needs one of these to compute.

Copies and congruence. For a native axiom $e = \texttt{true}$, NativeTrust copies every definition that the evaluation of e can reach and that needs a change. Inside the copies it replaces `WellFounded.fix` (Section 3.4) and any replacement registered by the user (Section 3.2) by versions the kernel can evaluate. The kernel then checks $e' = \texttt{true}$ by `rfl` for the copied term e', and $e = e'$ is proved by congruence from per-definition equations $c = c'$. Comparing an original closed term with its

copy directly could make the kernel evaluate both, so the converter does so only when congruence fails; for the per-definition equations it then runs under a heartbeat bound. When nothing needs a change, this is plain kernel evaluation of e.

3.1 Facts inside definitions and instances

Declarations such as

```
def cert := ⟨..., by native_decide⟩
instance : Fact (Nat.Prime p) := by native_decide
```

put a native axiom inside data that other facts mention. Such facts are proved first, compiled to their own modules, and reused by every later fact whose module imports them. A statement that mentions such a definition is restated over its rebuilt copy. When the rebuilt declarations are definitions or theorems, the two statements are definitionally equal by proof irrelevance, though not expression-identical; a rebuilt `opaque` constant does not unfold, so a statement over it would be a different proposition. The converter does not check this, but for the supplements a separate statement check does (Section 6). In the study, 342 facts sat inside definitions or instances.

3.2 Externally proved facts and proved replacements

A declaration `@[kernel_fact] theorem h : P` registers a proof of P. A native fact `decide` $P' =$ `true` uses it when P' is syntactically equal to P, or definitionally equal at reducible transparency. The proof is then `decide_eq_true` h, with the fact's own decidability instance, and the converter rejects a registered proof that uses axioms beyond the standard three. We provide Pratt certificates [28], checked through Mathlib's Lucas primality theorem and a structurally recursive modular exponentiation, for the field primes of common zero-knowledge and cryptographic libraries: BabyBear, KoalaBear, Mersenne31, Goldilocks, BN254 (both fields), secp256k1 (both), BLS12-381 r and Curve25519. The kernel checks a 256-bit certificate in 0.24–0.57 s. Eight corrupted certificates were rejected.

Similarly, a declaration `@[kernel_replace] theorem t : @f = @g` registers g as a replacement for f inside copies. The converter uses the body of g as it stands, so g must itself be evaluable.

3.3 Triage, bounded evaluation and splitting

A static pass lists the blocking constants that the evaluation can reach, with the chain of definitions that leads to each. It follows every constant in definition bodies, including higher-order arguments and instances, as the copies would. It does not enter theorems and skips panic and trace messages, which kernel evaluation does not reduce, but it follows constants inside the proofs that definitions contain, which evaluation never reaches. The accessibility proofs of the accessibility route in Section 3.4 are treated separately. The pass is designed to over-approximate, but we have not proved that it does. When it finds a blocking constant, evaluation is capped at 50,000 in the units of Lean's `maxHeartbeats` option ($5 \cdot 10^7$ heartbeats), because a stuck evaluation can exhaust memory before it fails. A capped run can fail for lack of budget and still count as blocked, so we re-ran a sample without the cap (Section 4.5). Checks of the form a && b or `List.all` l f are split into separate kernel declarations joined by small lemmas, which reduces the memory each declaration needs.

3.4 Well-founded recursion by fuel

```
def fixFuelAux (hwf : WellFounded r) (F : ∀ x, (∀ y, r y x → C y) → C x)
    (n : Nat) : ∀ x, C x :=
  Nat.rec (fun x => WellFounded.fix hwf F x) (fun _ ih x => F x (fun y _ => ih y)) n
def fixFuel hwf F x := fixFuelAux hwf F 18446744073709551616 x
theorem fix_eq_fixFuel : @WellFounded.fix = @fixFuel
  -- induction on n, WellFounded.fix_eq
```

The auxiliary function `fixFuelAux` unfolds the functional n times and then falls back to `WellFounded.fix`. The equation follows by induction on n from `WellFounded.fix_eq` alone; no accessibility proof is reduced and `Acc` is not eliminated into a type. Evaluation relies on the kernel's support for natural-number literals, which turns a literal $n > 0$ into `Nat.succ` $(n-1)$ when a recursor is applied to it. Hence `Nat.rec` on 2^{64} unfolds one step per recursive call actually made, and an evaluation that finishes never reaches the final `WellFounded.fix`. Both nanoda, with its natural-number extension, and con-ron accepted outputs that rely on this. The lemma applies to every definition compiled through `WellFounded.fix`, but whether congruence can then bridge the copy depends on the definition (Section 4.7).

Proofs inside copies. A copied body can contain proofs about the constants being replaced: `Multiset.sort` is `Quot.liftOn` with a proof about `List.mergeSort`, and `Nat.find` comes with `Nat.find_spec`. A copy equals its original only propositionally, so substituting inside such a proof makes it ill-typed. The converter keeps these proofs as they are and transports them along the copy equations (`Eq.mp`), and it proves the copy equation by dependent congruence, abstracting each replaced constant together with its equation. This runs as a retry of the fuel route, after a first attempt without it fails (Section 4.7). The retry also keeps the original of a definition whose copy would equal it (the unchanged-copy rule). Copies of definitions that also occur in types, such as an instance inside the type of another definition, cannot be bridged this way; a later route handles most of them (Section 4.8).

The previous version of NativeTrust used a lazily unfolding accessibility proof ($2^{64}-1$ layers of `Acc.intro`, as in Coq's `Acc_intro_generator`), bridged by proof irrelevance. It needs the kernel to reduce `Acc` proofs. Core PR #14892 (merged 14 October 2026, "define well-founded recursion without large elimination of `Acc`") [13] removes that reduction, and con-ron rejects the construction. When the fuel route fails even with the retry, the converter falls back to the accessibility route, which works up to Lean 4.36, and records the fact as `acc_fallback`.

3.5 Driver and supplement

The driver, `lake exe ntrust`, runs one process per fact under a wall-clock limit and an address-space cap, proves facts inside definitions and instances first, and writes three outputs. The first is a report with one line per fact. The second is a *kernel supplement*: one module per source module that names a native tactic, with a theorem T.`kernel` for every theorem T of that module that depends, directly or through other declarations, on a native fact, provided all those facts were converted; theorems of other modules get none. The third is the list of pairs $(T, T.\texttt{kernel})$ for Comparator. Except for the restatement over rebuilt copies, each T.`kernel` is declared with T's own type, so the kernel's type check is also the statement check. The static pass also serves as a linter (`set_option linter.nativeTrust true`), which warns when a native fact is written if the evaluation can reach a blocking constant. We have not measured its false-positive rate.

4 The study

4.1 Setup

Corpus. We examined 30 public repositories (Appendix A). They were the repositories with the most files that mention `native_decide` in code search, plus research developments, a corpus of AI-written formalizations, and software-verification projects. Sixteen packages contain native facts that build at their pinned toolchain (Lean 4.28–4.35); rs-mca has 8 sub-projects, which we count as one package. In our setup, 546 modules did not build: 263 in TrainVerify, 115 in sounio, 95 in ternary-goldbach, 39 in tetrasemaphore (no Lake file), 9 each in aks and sorting-network, and 16 in five others (FloatLib 6, lido-srv3 5, veil 3, formal-conjectures 1, and one in a sub-project of llm-rust-lean-pipeline). Facts in those modules are not counted.

Unit and limits. The unit is one native fact: one native axiom on Lean $\geq$4.29, or one use of `Lean.ofReduceBool` on Lean 4.28. A single tactic call can produce several facts (e.g. under `all_goals`). The per-fact driver measured 52,725 facts: 52,523 from `native_decide`, 197 from `decide +native` and 5 from `bv_decide`. An earlier per-theorem driver measured the other 96, in six rs-mca sub-projects. Each fact ran in its own process with a 900 s wall-clock limit and a 48 GiB address-space cap, with 8 to 32 processes in parallel; facts that failed for resources were re-run with 3,600 s and 96 GiB (the heavy pass). All runs used one 96-core cloud machine with 743 GiB of memory. The three packages with well-founded copies (cvc5/logos, Strata, formal-conjectures) were run with the current tool, formal-conjectures including the heavy pass. The other packages were run with builds made before the fuel route, which also differ from it in the triage, the part that decides where the 50,000-heartbeat cap (Section 3.3) applies. The regression runs that gated each later extension (Section 4.8) re-ran the converted facts of five of these packages and 2,018 of TrainVerify's with later builds and lost none.

4.2 Results

Table 1 lists the packages. Nine packages convert at least 90.7% of their facts; the other seven convert at most 38.5%. Per package, the median rate is 90.8% and the mean 59.3%. The pooled rate is 98.0% (51,764 of 52,821), but TrainVerify contributes 86.6% of all facts. Without it the pooled rate is 85.3% (6,057 of 7,101); without the three largest packages it is 65.9% (2,012 of 3,052). A converted fact took a median of 2 s including the import (90th percentile 3 s), with a median peak memory of 3.3 GiB. These medians mostly reflect TrainVerify. No evaluation in the corpus that finished reduced to `false`.

4.3 What stops the rest

Table 2 classifies the 1,057 failures. A failure is *traced* when a reachable blocking constant or a stuck construct was identified; the class names it. In all, 825 failures are traced and 232 are not.

Strings. The largest class comes from string processing at check time. BerryEsseen stores its cover certificates as strings and parses them inside the decided term: 569 of its 579 facts are blocked there, 8 hit resource limits and 2 convert. The 16 cedar-spec failures are parsing tests, whereas every native fact that cedar-spec uses inside proofs converts. The string functions involved loop through $\texttt{WellFounded.extrinsicFix}_3$, and their iterator state types are defined through an opaque constant, so copies of the iterator functions cannot be bridged by congruence. The remedy in Section 4.4 replaces the string functions one level up.

package	domain	facts	converted	rate
TrainVerify (distributed-training graphs)	software	45,720	45,707	99.97%
cvc5/logos (checker for cvc5 proofs)	software	2,185	2,185	100%
Humanfia IChO 2026 (AI-written)	chemistry	1,864	1,860	99.8%
formal-conjectures	mathematics	690	664	96.2%
longfellow-zk (Lean model)	software	690	626	90.7%
asi-stack-book	software	392	383	97.7%
rs-mca (8 sub-projects)	mathematics	107	104	97.2%
sounio	software	77	70	90.9%
lido-srv3 proof closure	software	70	70	100%
BerryEsseen	mathematics	579	2	0.3%
Strata	software	286	66	23.1%
clean (zk circuits)	software	120	17	14.2%
cedar-spec	software	26	10	38.5%
aks	mathematics	7	0	0%
sorting-network	mathematics	7	0	0%
aeneas	software	1	0	0%
total		52,821	51,764	98.0%

Table 1: Native facts converted automatically, per package. With the two remedies (Section 4.4), clean converts 116 and cedar-spec 26; with the `bv_decide` route (Section 7) as well, clean converts all 120 and asi-stack-book 384.

Casts. Thirteen facts stop at a cast. The decidability instances involved were built with `simp <;> infer_instance` (asi-stack-book), come from Mathlib's `Fintype` instance for the edge sets of simple graphs, or are derived for a quotient type (`Nat.Partition`). Their terms contain `Eq.rec` along an equality proved with `propext` or `Quot.sound`, whose two sides are not definitionally equal (Section 2). No constant is missing, so the static pass reported nothing. A diagnostic added after the study follows the kernel's weak head normal form into stuck recursors and names the cast and the definition that contains it. Since the study, a retry builds cast-free versions of such instances, and all 13 facts convert (Section 4.8).

The rest. Strata's checker runs through Lean's own parser and environment infrastructure. The six C-level failures load data through a project-specific `mmap` extern, call `Float` functions, or use `USize`. Of the 230 resource failures, 99 are clean facts that depend on primality instances (Section 4.4), and two longfellow-zk facts evaluate $\binom{m}{3}$ for $m \approx 10^7$ by unary recursion and exhaust the stack. For the resource failures the driver records no blocker: either none is reachable, or the run reached its limit before the static pass reported. Four of the five `bv_decide` facts are in this row although their LRAT parser is reachable, so the row should not be read as "no blocker reachable".

String literals. A literal is unfolded to `String.ofList`, whose bytes come from `List.toByteArray`, a left fold of `Array.push`; each `Array.push` appends to the end of the array's list. For prefixes of one of BerryEsseen's certificate literals, in files that import nothing, computing the UTF-8 size in the kernel takes 21 s and 4.4 GiB at 1,000 characters and 84 s and 15.6 GiB at 2,000 (with a raised `maxRecDepth`). A synthetic non-repetitive literal of 1,000 characters in a file that imports nothing behaves the same (19 s, 4.4 GiB), and so does a fold of `Array.push` over the same bytes without any string. Peak memory fits 0.45 GiB plus 48.6 KiB per byte plus 8.27 KiB per distinct substring of the bytes (64 runs, worst error 2.1%), so repetitive literals are cheap and

class (constants reached)	facts	packages
iterator-based string functions (`WellFounded.extrinsicFix`$_3$; iterator state types defined through `Std.Internal.idOpaque`)	585	BerryEsseen 569, cedar-spec 16
Lean's metaprogramming layer (`partial` parser, `PersistentHashMap`, unsafe implementations, `partial_fixpoint`), `HashMap`, `USize`	220	Strata
casts along equalities proved with `propext` or `Quot.sound`, inside decidability instances*	13	asi-stack-book 8, formal-conj. 5
C primitives and platform constants (an `mmap` data loader, `Float`, `USize`)	6	aks 2, sorting-network 2, sounio, aeneas
the LRAT parser of `bv_decide` (`partial`, hashing)	1	clean
time, memory or recursion-depth limit reached	230	clean 102, longfellow-zk 64, formal-conj. 21, TrainVerify 13, BerryEsseen 8, sounio 6, aks 5, sorting-network 5, Humanfia 4, asi-stack-book 1, rs-mca 1
not classified (earlier driver version)	2	rs-mca

Table 2: The 1,057 facts not converted. The first five classes are traced. *Found by a diagnostic added after the study run; the study's static pass reported no blocker for these.

non-repetitive ones quadratic; by this fit, BerryEsseen's 7,824-character literal would need about 240 GiB. Replacing only `List.toByteArray` by a version that builds the array in one step, proved equal to it, makes the cost linear: for `d.utf8ByteSize > 0` on the same literals, the kernel needs 23, 36, 67, 131 and 257 MiB at 500, 1,000, 2,000, 4,000 and 8,000 characters, against 1.0, 4.0 and 15.9 GiB at 500, 1,000 and 2,000 characters on the original path, and at most 1 s. Both sets of figures are measured above the 1.5 GiB of a file that imports Lean and does nothing else. The cause is therefore the `Array.push` fold in `List.toByteArray`. Core's string-equality simprocs were changed in March 2026 so that their proofs no longer make the kernel run the UTF-8 encoding [47], but other evaluations of literals still take this path.

4.4 Two remedies

Primality inside instances. Of clean's 102 resource failures, 99 depend on instances `Fact (Nat.Prime p)` proved by `native_decide`, and their checks run out of memory. The other three are `bv_decide` facts. We registered four `norm_num` primality proofs with `@[kernel_fact]`, one for each prime the package uses. All 99 then convert, and clean goes from 17 to 116 of 120; its four `bv_decide` facts convert by the kernel-checked route (Section 7).

String functions. We wrote list-based versions of the string functions above the iterators, `String.toNat?` and `String.toInt?`, proved equal to the originals from core lemmas such as `String.Slice.foldl_eq_foldl_toList`, and registered them with `@[kernel_replace]`. The two cedar-spec wrappers around them also call `String.contains`, which is generic in its pattern (see below), so we replaced the wrappers too. All 16 cedar-spec facts then convert, in 7 s, and nanoda (6,429 declarations) and con-ron (5,964) accept the result, while nanoda rejects the original. These replacements do not reach BerryEsseen, for two reasons. It tokenizes with `String.replace` on a string pattern, which is generic in the pattern's searcher, and core has no lemma that characterizes

it by lists. It also stores its certificates in string literals of up to 100 KB, and evaluating a literal costs time and memory quadratic in its length, as described above.

4.5 Validation and ablation

Do the blocked labels hold without the cap? We re-ran facts labelled blocked with the cap switched off: random samples of 20 from BerryEsseen and 20 from Strata, and all 16 from cedar-spec. All 56 still fail. The BerryEsseen facts fail at once (about 1 s per fact in the driver's record, 5–10 ms of it in the kernel), with the kernel stuck on the same opaque constant.

What do the transformations add? We re-ran 51,329 of the 51,425 converted facts outside definitions and instances, with well-founded replacement and splitting switched off. The 96 left out are rs-mca's 94, converted by an earlier per-theorem driver, and two formal-conjectures facts that converted only in the heavy pass. This is close to applying `decide +kernel` to each fact, except that facts inside definitions and instances are still rebuilt first. In this run, 51,307 converted again. The 22 that did not are six TrainVerify facts that need splitting and 16 facts blocked at `WellFounded.fix` (one in cvc5/logos and 15 in formal-conjectures). Without splitting, five of the TrainVerify facts are capped and run out of budget, because the static pass finds a constant of Mathlib's real numbers reachable from the whole check, and the sixth runs out of memory. Since the kernel cannot unfold `opaqueId`, a fact that converts without the replacement never evaluates a definition compiled through `WellFounded.fix`. All 1,705 converted facts with well-founded copies were also run without the replacement, and 16 failed. One of the 1,705 lies inside a Strata definition and was run as part of its module. One more conversion, by rs-mca's earlier per-theorem driver, also made copies: it fails without the replacement and converts on the fuel route. The other 50,058 conversions made no such copies, including the two that were not re-run.

For most facts, then, the converter transforms nothing. It applies kernel evaluation to each fact as written, proves facts inside definitions and instances first, bounds and diagnoses failures, and writes a supplement whose theorems use only the standard axioms.

4.6 A random sample of repositories

The corpus is a selection that favours heavy users. As a check, we sampled from the 585 repositories that GitHub code search returned for `native_decide language:Lean` in October 2026. We excluded 35 repositories we had examined or inspected before (24 of the 30 in the corpus and 11 others) and 40 repositories whose size, as GitHub reports it, is at least 400,000 KB. There were no forks or archived repositories. From the remaining 510 we drew 70 with a fixed seed (20261023). Of the six corpus repositories not excluded, two are not in the frame (Humanfia's and alphaproof-nexus) and four could be drawn. Only cedar-spec was drawn, and its two runs, in the study and in the sample, agree (10 of 26 facts converted). Packages pinning Lean older than 4.27 are not supported and were skipped (14 packages in 9 repositories). The others were run with the same driver and limits, except WyattAu/clawdius: it names its toolchain 4.28.0, without the usual v, and our scripts did not run it; its one module with native facts has three uses of `native_decide`. Native facts that build occur in 26 packages, 2,018 facts in all. The per-package median is 77.8% and the mean 59.0%. The pooled rate is 44.9%, or 57.5% without our own package, which dominates it: 409 of its 534 facts stopped at a resource limit. Most of the 1,111 failures (827) hit memory, time, stack or recursion-depth limits, and three packages account for 706 of these resource failures. The traced ones reach `HashMap/USize` (114), iterator-based string functions (73) and Lean's metaprogramming layer (74). In three packages some copies do not bridge, because a copied definition occurs in the type of

an argument (Zag 3 facts, BobDyLean 13, minidregg 1). With the fuel route's retry, Zag converts its other 20 facts. Three of minidregg's conversions use the accessibility fallback (Section 3.4); with the current library they convert without it. One fact, in a test module for compiled overrides, evaluates to `false` in the kernel; NativeTrust reports it and produces no proof.

4.7 Robustness to core PR #14892

Among the evaluations in the study, core PR #14892 can affect only those that run well-founded recursion. The copier made well-founded copies for 1,706 converted facts (cvc5/logos 1,652, Strata 37, formal-conjectures 16, rs-mca 1), all but one for the projects' own functions, and 17 of them evaluate the recursion (Section 4.5); the rs-mca fact copies core's `List.merge` and `List.mergeSort`. Without proof transport, the fuel route converts cvc5/logos's one such fact, but congruence cannot bridge the copies of the 15 in formal-conjectures, which then convert only through the fallback. The current tool retries the fuel route with proof transport (Section 3.4), under fresh copy names, when the first attempt fails and well-founded recursion is reachable, and only then falls back. With the fallback switched off, all 15 convert. Facts that converted on the first attempt are unaffected. Re-running gave the same 66 conversions for Strata, all 2,185 for cvc5/logos and, in a first pass without the heavy pass, 662 of 690 for formal-conjectures. None of them used the fallback. With the current tool, then, 51,747 conversions never evaluate a definition compiled through `WellFounded.fix`, and the other 17 do so on the fuel route; none depends on the accessibility route. (The rs-mca fact converted through the accessibility route in the study; a re-run on the fuel route with the fallback switched off converts it.)

The study packages pin Lean 4.28–4.35 and cannot be built on the nightly, so the nightly test is on two functions. On `nightly-2026-10-22`, which contains PR #14892, the fuel route converts `List.mergeSort` and a lexicographic Ackermann function. Run alone on mergeSort, in a file of its own, the accessibility route fails there, stuck at `Classical.choice`, while it succeeds on Lean 4.34. Plain `decide +kernel` fails on both toolchains.

4.8 Closing the blocked classes

After the study we extended the library and re-ran the failures. The extensions for casts, copies in types and string functions run only after a fact's first attempt fails, so a fact that converted before converts the same way; the linear evaluation of string literals of 128 or more characters and the extensions for large checks change the first attempt itself. Each extension was accepted only after every previously converted fact of nine packages and 2,018 of TrainVerify's (about 7,300 facts) converted again with no fallback.

Resources. A cheap term is now tried whole within a small budget before any search for structure, and each search step has its own budget: in 569 of the 638 time and memory failures of the corpus (228) and the sample (410) outside our own repository, the meta-level search for split points, not the kernel, had run out of time, memory or recursion depth. Existentials are checked at the witness that compiled code proposes. Optionally, conjunctions are split, a negated conjunction or a disjunction is checked at the part that compiled code names, and bounded universals are checked by index. Finally, a check can be sharded into modules compiled in parallel, each proving one piece under a module-independent name, and a module that checks every imported statement joins the pieces. We re-ran 398 of the 638 with these changes; the others are 117 `bv_decide` facts, clean's 99 primality facts, and 24 longfellow-zk and BerryEsseen facts that ran out in the kernel itself and went straight to sharding. With the study's limits, 242 of the 398 convert (234 with the defaults or the structured

checks, 8 sharded, up to 5,943 pieces). In total, 388 of the 638 now have kernel proofs: these 242, the 99 primality facts with their `norm_num` proofs (Section 4.4), the 24 sharded facts, and 23 facts re-run with 3,600 s and up to 96 GiB. The 117 `bv_decide` facts convert by the kernel-checked route (Section 7). Of the other 133, 31 convert with the options and add-ons below, and most of the remaining 102 are beyond kernel evaluation as it stands. In 34 of these 102, a decision procedure builds an array by repeated `push`, and since an array is a list in the kernel, each push copies it: one cSHAKE circuit of 607,242 gates takes about $1.8 \cdot 10^{11}$ list steps to build. Eighteen decode a serialized term of 566 KB (about 10^{13} steps), and four perform 2,048 scalar multiplications of about 12 s each. Twenty-one longfellow-zk facts hit the recursion-depth limit when re-run; they evaluate Mathlib's binomial coefficient by Pascal's rule, about 8.7 million levels deep, inside decided propositions. An add-on proves Boolean forms of the two functions that decide them, using a fast binomial coefficient, and all 21 convert, three of them also with the opt-in options below. The study's two longfellow-zk facts that exhausted the kernel's stack (Section 4.3) were not re-run. Of four facts that test primality, three convert with an opt-in option and one with a Pratt certificate. Of the 48 facts with a single piece above 48 GiB, 29 convert when re-run with 3,600 s and up to 200 GiB per piece, and 23 of the 29 stay within 96 GiB.

Casts. When the kernel gets stuck at a cast inside a `Decidable` or `Fintype` instance, the library builds a cast-free version of the instance (by splitting the proposition with `casesOn`, by `decidable_of_iff` over the fields of a structure, by `Fintype.ofFinset`, or by turning the cast into `decidable_of_iff`), proves it equal to the original by `funext` and `Subsingleton.elim`, and retries. Both are kernel-checked, and any value of a `Decidable` or `Fintype` type is correct by construction, so nothing new is trusted. All 13 cast facts of the study convert. In formal-conjectures' copy of Mathlib, 61 of the 84 such instances that hold a cast of data get a cast-free version.

Copies in types. The fuel route's retry no longer copies type formers and proofs, proofs nested inside types are transported, and a parametric route rewrites definitions that occur in types together with those types. Copies in types blocked 17 facts. Now 16 of them convert (BobDyLean's 12 with a package-specific `DecidableEq` instance), and the seventeenth is no longer blocked but is too slow. All 23 facts of Zag now convert.

Strings. Copies evaluate long string literals through the proved one-step form (Section 4.3). A library module gives proved list-level versions of the iterator-based `String` functions that the failing facts use (`toNat?`, `foldl`, `replace` with a string pattern, `contains`); a function generic in its pattern is replaced for one pattern type by first folding its applications into a specialized constant, which is definitional. These replacements apply only in a retry: applied in the first attempt, they made Strata copy formatting code that its facts reach but never run, and 32 of its 66 facts timed out. As a retry, they convert 17 of the 42 iterator-string facts outside BerryEsseen, cedar-spec and our own package, and all 16 of cedar-spec's without package-specific replacements. A package add-on for lean-regex replaces its quadratic pair checks by linear ones and converts its two facts. The remaining 23 iterator-string facts reach partial functions, which have no equations in the logic: Lean's JSON parser and compressor in 21 and a package's own predicate in two. BerryEsseen now converts all 579 of its facts (2 in the study) with a package-specific tokenizer replacement, its covers parsed once per certificate, and the 8 sharded chunks above. These 577 conversions took 742.5 CPU-hours, and no process needed more than 42.7 GiB.

Platform word size. `USize`, and through it `HashMap` and `HashSet`, depends on the opaque word size `System.Platform.getNumBits`, but core proves that it is 32 or 64. A retry evaluates a term that reaches it once for each value and joins the two kernel proofs by cases; a result that differs between the two word sizes is not proved. Because the kernel computes with numerals only in closed terms, each case is first checked on a copy in which the hypothesis on the word size is replaced by a closed stand-in; the copy never enters a proof. The retry converts all 14 facts of ninegates, two of them only with the 3,600 s limit. Hash maps keyed by strings stay blocked, since `String.hash` has no definition. Package add-ons convert Strata's 34 such facts, by proving lookups equal to scans of an array that Strata keeps next to the map, and five of minidregg's, by proving a list version of its graph linearization equal to the original.

Opt-in options. Two further options stay off by default. With `stage`, intermediate values of a computation are computed by compiled code, proved one by one in the kernel as separate lemmas, and joined by a kernel-checked bridge, so that no single declaration evaluates the whole computation; a wrong value only makes a check fail. With `csimp`, the proved equations that Mathlib registers for its compiled code (a product formula for binomial coefficients, trial division up to the least factor, binary-splitting factorials) serve as replacements. Together they convert eight more facts: two in sounio by staging, three in longfellow-zk by the replacements with structured checks, one each in formal-conjectures and goya by the replacements alone, and one in TrainVerify with all of them and a 3,000 s limit. In these runs the staged route did not yet check the axioms of its final theorem; we checked the five facts that went through it (the two in sounio, the one in TrainVerify, and two btc facts that the re-runs with more memory also convert) with `#print axioms`, and all use only the standard axioms (version `6b378a8e` of Appendix B adds the check). The other 40 facts that they convert had already converted by other means. Staging does not reach the large computations above: in most of the 80 facts that compare large computed values, the staged parts are themselves too expensive.

The random sample. Some of these extensions were developed against the random sample's failures, so the sample does not test them on unseen packages. With the library alone, its pooled rate rises from 44.9% to 58.5% and its per-package median from 77.8% to 100%; with the `bv_decide` route the pooled rate is 68.8%, with the opt-in options 68.9%, and with the package add-ons 69.8%.

5 Removing unnecessary `native_decide`

The ablation (Section 4.5) leads directly to a tool. Importing `NativeTrust.KernelFirst` registers elaborators for `native_decide` and `decide +native` that Lean tries before its built-in ones. Each call first runs the static pass and, if no blocking constant is reachable, builds the same auxiliary lemma as `decide +kernel` and lets the kernel check it within a heartbeat budget (2,000 in `maxHeartbeats` units by default). If the kernel proves the goal, that is the proof, with no native axiom. Otherwise the state is restored and Lean's own native path runs, so the call behaves as before, and an info message gives the reason. No call is edited. The kernel attempt adds a declaration that Lean's kernel checks; nothing else enters the proof.

On unmodified sources, adding only this import removed every native axiom from lido-srv3 (723 modules, 70 to 0) and from cvc5/logos's 10 largest modules (1,043 to 0). Building took about the same time with and without the import: 343 and 345 s for lido-srv3, 187.6 and 184.7 s for the cvc5/logos modules. Where kernel checks are expensive it costs more: codegraff, a small package from the random sample, took 17 s instead of 2.6 s. The module imports Lean, and through it

package	native axioms before	after	defaults build wall time (s)	after	without limits build wall time (s)
lido-srv3	70	0	7.1 → 7.8	0	7.8 → 15.6
cvc5/logos	2,185	3	1,740 → 1,693	1	1,855 → 1,717
Humanfia IChO 2026	1,864	194	93.6 → 95.3	145	94 → 470
asi-stack-book	392	10	22.2 → 21.9	10	29.7 → 28.7
sounio	77[a]	35	4.3 → 4.2	7	6.8 → 205
formal-conjectures	690	126	31.6 → 33.9	111	45 → 211
total	5,278	368		274	

Table 3: Rewriting `native_decide` as `decide +kernel` (`ntrust migrate`) in six corpus packages, with the default limits (a per-fact budget of 2,000 and a slowdown limit of 2) and without them, in separate runs; both runs also limit the time of a build round. Build times are clean rebuilds of the affected modules, with the same job count before and after. With the defaults 2,058 of the 2,339 uses (examples included) were replaced, without the limits 2,220. [a]664 further uses are in modules the original build does not build. Without the limits and with heartbeat limits lifted for four declarations, Humanfia goes to 1,864 → 4. Both runs remove all 70 axioms of lido-srv3; with the defaults the slowdown limit kept the native examples of two of its modules.

Lean's `Std` library, into every importer. In codegraff, six structure literals written in the shorthand `{ a, b }` were then elaborated as set notation, because `Std` has generic `Insert` and `Singleton` instances; importing Batteries, Aesop or Mathlib has the same effect, and a type ascription fixes each literal. For projects that cannot take that import, `NativeTrust.KernelFirstLite` is the same switch as two macros that import nothing beyond Lean's prelude, without the static pass, options or reports: codegraff builds unchanged with it (88 to 23 native axioms), and lido-srv3 loses the same 70 axioms. Its build then takes 421 s instead of the 360 s of that run's unmodified build (343 s in the run of the full switch), because a call that the kernel cannot check spends the budget before it falls back.

For projects that want no dependency on NativeTrust, a source rewriter (`ntrust migrate`) replaces by `decide +kernel` each `native_decide` whose facts plain kernel evaluation proves within a per-fact budget (2,000, as for the switch, by default), so expensive facts stay native. AXLE's `repair_proofs` [38] makes the same replacement within one submitted file. `ntrust migrate` instead works on a whole project: it rewrites the sources in place, keeping the originals, and rebuilds the project. It reverts each replacement that fails, that leaves a module building past a round's time limit (900 s by default), or that makes a module's build more than twice as slow, and then checks the remaining axioms. The per-fact budget and the slowdown limit can be switched off, but the time limit cannot. Table 3 shows six corpus packages with the default limits and without them. Every rewritten package builds, and the native axioms that remain are exactly those of the facts whose uses were kept. Without the limits, build time fell on cvc5/logos, where compiling the decision procedures for `native_decide` cost more than the kernel checks. It was level on asi-stack-book and rose where single facts are expensive to evaluate in the kernel, about thirtyfold in sounio. With the defaults no build took more than 1.1 times as long, and 368 instead of 274 native axioms remain.

To measure this without the selection of the corpus, we ran both tools on every package of the random sample except our own (Table 4). The rewriter, run without its limits, removed 54% of the native axioms, 76% in the median package. Of the 683 native axioms that remain, 208 come from `bv_decide`, which a rewrite to `decide +kernel` cannot handle (they convert by another route, Section 7). Among the rest, 120 belong to facts too expensive for plain kernel evaluation within the

	native axioms removed	uses replaced	builds	build time
`ntrust migrate`, defaults	30.1% / 25.0%	28.1% / 47.2%	23 of 24[a]	0.99 / 1.00
without budget or slowdown limit	53.6% / 75.9%	50.0% / 80.7%	23 of 24[a]	5.4 / 1.9
one-import switch, budget 2,000	35.5% / 27.3%		21 of 24[b]	1.3 / 1.2
budget 20,000 where needed	48.4% / 39.8%			

Table 4: Removing native axioms from the random sample: 24 packages (all with native facts except the two of our own repository), 1,472 native axioms and 2,361 uses, with the extended library (Section 4.8). Pooled rate / per-package median; build time is the ratio of clean-build wall time after to before, pooled / median. [a]The 24th, zcash, fails only in its own checks that require some theorems to keep a native axiom. [b]zcash and minidregg fail only in such checks; in codegraff, six structure literals become set notation once `Std` is in scope, and it builds with `KernelFirstLite`.

per-fact limits, 101 to rewrites that were reverted because their modules took too long or too much memory to build, 80 to modules whose own checks require the native axioms, and 174 to other causes, mostly constructs that plain kernel evaluation cannot handle (iterator-based strings 58, well-founded copies 58, `HashMap` or `USize` 20, the metaprogramming layer 16, casts 10, other 12). Six of the 24 packages check axiom footprints in their sources, and four require some theorems to keep native axioms (one of them only `bv_decide` axioms), for instance with `#guard_msgs in #print axioms` or with a command that fails when a theorem marked as native "reaches no native_decide axiom". In those projects removing a native axiom is a change their own tests reject, and the variants built without the audit modules all succeed. That rewrite multiplied the median clean-build time by 1.9 and the pooled one by 5.4. One small package took 143.5 times as long (1.34 to 192.29 s), because its rewritten facts are expensive in the kernel. With the defaults the rewriter removed 30.1% of the native axioms (25.0% in the median package), and no package's build took more than 1.7 times as long; the slowest went from 1.03 to 1.74 s. The budget keeps expensive facts native, and the slowdown limit reverted modules in six packages where many cheap facts added up. The one-import switch removed 36% at the default budget and 48% with a ten times larger budget where axioms remained, at 1.2 to 1.3 times the build time. It fails on the two packages with axiom checks and on codegraff.

For new code, the linter (`set_option linter.nativeTrust true`) evaluates each new native fact the same way and, when the kernel proves it, warns that `native_decide` is unnecessary, with a "Try this: `decide +kernel`" fix (Mathlib's linter [46] flags every use instead); a strict option makes the warning an error for continuous integration, and `#native_decide_audit` prints a verdict for every native fact of a module. On cvc5/logos and lido-srv3 modules the linter added at most 4.4% to elaboration time.

6 Independent checks

Method. We export kernel theorems with their full dependency closure, using `lean4export` [25] built for the package's toolchain. We then run two checkers distributed with the Lean 4.35 toolchain: nanoda 0.4.19 [26], a separate implementation of the kernel, with the three standard axioms permitted and any other a hard error; and con-ron, which never reduces proofs. An empty export counts as a failure. We report inputs that con-ron does not support as declined. Neither checker compares T.`kernel` with T, so for the supplements we also run a separate statement check. For every T.`kernel` in a supplement module, a fresh Lean process compares its type with T's by expression equality; when they differ, it adds a theorem with T's type and the value T.`kernel`, which Lean's

package	facts	theorems	modules	same / defeq statement	nanoda	con-ron
cvc5/logos	2,185	1,115	205	1,115 / 0	205	204 (1 declined)
lido-srv3	70	84	9	84 / 0	9	8 (1 declined)
Humanfia IChO 2026	1,860	395	34	375 / 20	34	34

Table 5: Kernel supplements, one kernel theorem for every theorem whose native facts all converted, in the modules that name a native tactic. Facts converted; kernel theorems produced; supplement modules; statements expression-identical / only definitionally equal (all 20 confirmed by the kernel); modules accepted. Humanfia has 407 theorems with native facts; 12 depend on its 4 unconverted facts. Producing the supplements took 25 minutes (cvc5/logos, 24 jobs), 15 s (lido-srv3, 8 jobs) and 24 minutes (Humanfia, 16 jobs); the independent checks of the cvc5/logos supplement took 70 minutes on 8 jobs.

package	sample	nanoda (declarations)	con-ron (declarations)
cvc5/logos	40	5,455	4,749
Humanfia IChO 2026	40	7,435	6,368
Strata	20	13,868	12,359
lido-srv3	30	1,847	1,412
cedar-spec	10	719	632
rs-mca (m31)	9	663	548
asi-stack-book	30	2,613	declined
sounio	30	1,953	declined
longfellow-zk	30	4,920	4,385
formal-conjectures	30	4,839	4,367
clean (with the four primality proofs)	30	4,249	3,796
TrainVerify (one fact from each of 24 module groups)	24	24 groups accepted	24 groups accepted

Table 6: Independent checks of random samples of converted facts, made with `b0da4454` (Appendix B). The cvc5/logos exports contain copies built on `fixFuel` and the lemma `fix_eq_fixFuel`.

kernel accepts only if the two types are definitionally equal. It also lists any axioms beyond the standard three, and as a positive control it confirms that each original T depends on a non-standard axiom. We did not run Comparator.

Supplements. We ran `lake exe ntrust` on cvc5/logos, lido-srv3 and Humanfia, and checked every module of each supplement separately, because some packages' modules cannot be imported together. Table 5 gives the results. In all three, every T.`kernel` uses only the standard axioms and every original uses a native axiom. The two modules con-ron declined exceed its expression arena (cvc5/logos) or use the `Nat.div` and `Nat.mod` definitions of Lean 4.31, which it does not support (lido-srv3). For one Humanfia module con-ron ran out of address space at 64 GiB and accepted it at 200 GiB, with a 30 GiB peak.

Samples. For the other packages we proved a random sample of converted facts again (seed 20261023) and checked them the same way (Table 6). The sample sizes were chosen per package to keep each export within the checkers' memory. Both checkers accepted every sample they could read. The checker con-ron declined two packages, for the same two reasons as for the supplements. The two checkers count declarations differently, and we report the counts as printed.

7 Kernel-checked `bv_decide` facts

A `bv_decide` fact states that an LRAT certificate, stored as a string, checks against the CNF of a bit-blasted goal. The certificate parser uses `partial` functions and the CNF construction hashes with opaque constants, so the kernel cannot evaluate these facts as they stand. We keep the user's theorem and replace the native fact inside its proof by a kernel-checked proof that the bit-vector expression is unsatisfiable (core's `BVLogicalExpr.Unsat`), which is what core's reflection lemma derives from the native fact. The new theorem is declared with the original theorem's type, so the kernel's type check is also the statement check.

Core's bit-blaster does not suit kernel evaluation: its and-inverter graph is built with arrays and hash maps, which the kernel evaluates as lists. On a 16-bit goal with 312 graph nodes, bit-blasting alone took 34 of the 69 s that core's pipeline needed to build the CNF. We wrote a bit-blaster for the kernel instead. It works on lists by structural recursion, builds each repeated word operation once (so x/y and x mod y share one divider), and emits Tseitin clauses over node indices. Its soundness theorem states that if this CNF is unsatisfiable, so is the expression; it covers every operation of `bv_decide`'s expression language, including bit reversal, counting leading zeros and population count.

An external SAT solver (CaDiCaL) produces an LRAT certificate for this CNF, which is stored as definitions and is not trusted. The original and the learned clauses sit in one balanced tree. The kernel first checks the original clauses against the bit-blaster in an in-order pass, then the certificate's steps. For large goals the first pass is split: one declaration runs the bit-blaster, one per segment of 30,000 graph nodes checks well-formedness and agreement, and proved lemmas join them. Each clause and the partial assignment are pairs of natural-number bitmasks, so unit propagation uses `Nat.land`, `Nat.lor`, `Nat.xor`, shifts and subtraction, which the kernel evaluates with GMP. Each step carries its unit literal, computed outside the kernel and checked with a subset test, because `Nat.log2` is not accelerated. The steps are checked in chunks of at most 20,000 hints, one declaration each, and the chunks can go into separate modules compiled in parallel. The soundness proofs use core's definitions and lemmas; they depend only on the standard axioms (checked on Lean 4.31, 4.33 and 4.35).

All five `bv_decide` facts of the study convert (Table 7). The largest, clean's ByteDecomposition `soundness`, has 65,720 clauses and a certificate with 959,442 hints; its kernel checks take 247 s (261 s with solving and preparation) in a process that peaks at 7.8 GiB. Split into four modules compiled in parallel, they take 113 s of wall time. We also ran every `bv_decide` fact of the corpus and the random sample (Section 4.6), and all 213 convert. They come from six packages (VeIR 188, Radix 17, clean 4, two packages of Natuvea 3, asi-stack-book 1); three convert only as modules compiled in parallel and four only with the split first pass. Radix pins Lean 4.29, which the current version of the route does not support. Its 17 facts convert there with two earlier versions: the one used for this sweep, whose soundness proof built on Lean 4.29 after a small change and uses only the standard axioms, and one that evaluates core's own bit-blaster in the kernel. Each converted fact depends only on the standard axioms, checked in a fresh process, and each rebuilt declaration has the statement of the original. Where theorems use several facts converted in different modules or packages, those facts are converted together, in one module, and after these joint runs every rebuilt declaration depends only on the standard axioms. The checker nanoda accepts a sample of them, including VeIR's `Veir.RISCV.Combine` rebuilt over 30 converted facts (29,187 declarations), and rejects the native originals. A `bv_decide` fact inside a definition yields a rebuilt definition without the axiom, but statements that mention the original definition, rather than the rebuilt one, still depend on it.

On synthetic goals the first pass costs about 0.5 ms per clause, and the steps about 0.22–0.25 ms

goal	clauses	hints	kernel time	peak memory
clean `not64_eq_sub`	2	2	0.05 s	2.6 GiB
clean `And8` / `Or8`	1,187 / 1,205	7,503 / 5,319	1.7 / 1.5 s	2.9 GiB
asi-stack-book	1,424	840	0.8 s	1.6 GiB
`admitted_layer_contract_is_complete`				
clean ByteDecomposition `soundness`	65,720	959,442	247 s	7.8 GiB
$(x \,\&\, y) + (x \mid y) = x + y$, 1,024 bits	73,697	232,083	191 s	
8,192 bits	589,793	1,863,909	2,054 s	47.9 GiB
$x \cdot y = y \cdot x$, 9 bits	3,161	5,865,891	1,352 s	
$x \cdot (y \cdot z) = (x \cdot y) \cdot z$, 7 bits	3,587	9,040,986	2,006 s	
$(x/y) \cdot y + x \bmod y = x$, 12 bits	8,909	5,639,962	1,380 s	

Table 7: Kernel-checked `bv_decide` goals: the study's five facts, then synthetic goals. Kernel time is the first pass plus the certificate steps (single first-pass declaration); one core, on a shared machine. Peak memory is the maximum resident size of the process and is given for the rows where the process checked one goal. All five study facts were blocked in the study.

per hint on the multiplication and division goals of Table 7, where steps carry many hints. With the first pass split, a goal with 1.18 million clauses converts at 20.4 GiB; the declaration that runs the bit-blaster is then the limit, at about a million graph nodes by extrapolation. Certificates of about ten million hints take 30–40 minutes; larger ones exhaust memory in the elaborator, which holds the parsed certificate, before the kernel starts.

A drop-in tactic. The same route is also a tactic, `bv_kdecide`, that can be written instead of `bv_decide`. It runs `bv_decide` on a copy of the goal in a scope whose changes to the environment are discarded, takes the reflected expression from the resulting proof term, proves its unsatisfiability as above and closes the goal with core's reflection lemma; no native axiom is added. Of the 208 `bv_decide` axioms that the rewriter left in the random sample (Section 5), 190 convert this way (VeIR 187, Natuvea 3). VeIR's last one is proved by the project's own tactic, and Radix pins Lean 4.29, which the add-on does not support. VeIR calls `bv_decide` through a macro of its own, at 247 call sites; changing that macro alone made one proof module build with 68 kernel-checked uses and no native axiom. On 195 facts of four packages the tactic took a median of 2.0 s per fact (90th percentile 53 s, at most 48 minutes and 16.8 GiB). With a limit of 60 s per fact, a rewrite would keep 176 of the 195 for 1,018 s of added build time.

8 Our own native results

Many of our own published results use `native_decide`. For most of them we later built kernel-checked versions by hand: we rewrote the decision procedures so that the kernel could evaluate them, generated check modules from the certificates, and built these modules in shards. The build records of these hand-built versions, for 23 of our results, add up to about 1.03 million CPU-hours. Nineteen of them prove the published bounds, and four only a weaker bound or a part. The artifact lists them with their sources. We ran the current library on the native versions at the study's limits, re-ran the facts that ran out of time or memory at 3,600 s and 96 GiB, and compared the outcome with the hand-built versions (Table 8). Where a class of facts was too large to run in full, we ran a sample. For nine results we ran the library's tactic `kdecide`, which takes the same route as the driver, on the facts' statements in copies of their modules, with compiled code switched off.

result	native facts	facts converted	theorems converted	CPU-hours native	CPU-hours NativeTrust	CPU-hours hand-built
with a hand-built kernel version						
QEC threshold, Theorem 1.1[a]	61	61	1 of 1	0.07	1.68	1.61
QEC threshold, all theorems	634	77[b]	1 of 16	17.2	9.4[b]	715.6[c]
$S(13) \geq 45$	1,724	0[d]	0 of 1	1,185[e]	2.2[d]	8,805.9
six-body central configurations	546[f]	56	18 of 97	–	11.1	1,178.3
3SUM/APSP lower bounds	11	11	14 of 14	<0.01	0.01	0.01
Hamming-cube Poincaré constant	3	2	0 of 4	<0.01	1.26	0.33
nine more results[g]	75	3	0			
without one						
PPT^2 counterexamples, core	751	540	543 of 770	0.70	20.1	–
PPT^2 counterexamples, rows	1,039[h]	271	304 of 1,219	4.11	27.0	–
four more results[i]	22	0	0			–

Table 8: NativeTrust on our own published results that use `native_decide`. Native: the native check modules alone for Theorem 1.1, 3SUM/APSP and the Hamming cube, otherwise the whole native build, measured on our machine except [e]as recorded in the release; hand-built versions as recorded in their releases, except [a]where all columns were measured on the same machine. [b]Including Theorem 1.1's facts; 525 of the 541 search facts were not run after the 16 sampled ones timed out, and NativeTrust's CPU-hours cover the facts that were run. [c]The kernel part, for Theorems 1.2 to 8.1. [d]22 probe runs, and the static pass on one fact of each class. [f]In the modules that build; the native headline theorem has 3,158. [g]Facts run, from the Zhang–Zagier bound and the eight results named in the text. [h]679 run, those of the cited theorems first. [i]Facts run.

Where the conversion replaces a hand-built version. Theorem 1.1 of the quantum error-correction paper [32] has 61 native facts, and all of them convert, in 1.68 CPU-hours. The supplement's theorem is expression-identical to the original and uses only the standard axioms, and nanoda checked the 21,775 declarations of its export. The checker con-ron, which accepted the export of an earlier conversion of the theorem (20,830 declarations), aborted on this one after a failed memory allocation at a 262.6 GiB peak. On the same machine and toolchain, the native checks of the theorem took 0.07 CPU-hours and its hand-written kernel version 1.61. That version evaluates the same checking functions with `decide +kernel`: two generators of 124 lines in all write its 390 check modules and their aggregator, and a 28-line file states the theorem. The conversion needs none of these files and takes 4% more CPU time. Our result on fine-grained lower bounds after the refutation of 3SUM and APSP [35] checks each of its 11 certificates twice: natively, and with `decide +kernel` on a checker written for the kernel, which a 163-line file proves equal to the native one. With the kernel theorems and their aggregator, the kernel version has 408 lines of its own. NativeTrust proves the 11 native facts about the original checker, so all 14 native theorems convert. Its runs took 52 s in all, against 42.8 s for the hand-built kernel modules and 2.4 s for the native ones (sums of per-module wall times). The rewrite was not needed.

Where it does not. The quantum error-correction paper's 15 other theorems use Theorem 1.1's facts and 573 more. Of the 573, 16 convert and 16 run out of 96 GiB (10 type tables and 6 factor bounds). Three of the theorems, Theorem 1.3(a) and two variants of Theorem 8.1(a), miss only their type table. The other 541 facts search over heads and covers of the decoding graph. We ran one from each of their 16 families, and all timed out, so we count the other 525 as unconverted. Without a time limit, the first head fact, which covers 342 heads, finished none of its eight pieces in

four hours, and its first head alone ran for an hour without finishing. The hand-built version checks the first 389 heads in 1,673 s, 4.3 s per head. The smallest cover fact ran out of 96 GiB after 470 s, while the hand-built module that checks the same basis and distance, for all five noise models, takes 13 s. The static pass finds `Float.log`, `mixHash` and the word size reachable from all 400 head facts. For these theorems the hand-built version rewrote the checks in 39 hand-written files (6,448 lines) and four generators (484 lines) that write 4,738 modules, and 662.3 of its 715.6 CPU-hours go to the heads.

None of the 1,724 facts of our proof of $S(13) \geq 45$ converts. The static pass, run on one fact of each class, finds blocking constants in the classes that hold 1,684 of them: their checker keeps memo tables in `Std.HashMap`s keyed by byte arrays, whose hash functions `ByteArray.hash` and `mixHash` are C primitives without a definition, and the root chunks also loop with `while`. Every fact reads its certificate data from string literals of up to 64 MiB. The one-step form of Section 4.3 starts from the literal's list of characters, a term as deep as the literal is long, and in all 22 probe runs, building it for the first literal overflowed the 1 GiB stack. With a 32 GiB stack, the kernel accepted the copy of one fact's last data chunk (4.8 MiB, in 18 s) but rejected each of its fifteen 64 MiB chunks for recursion depth. After 1,686 s of copying, the first piece of the check ran out of 96 GiB. Natively the module builds in 3.0 s. Even the project's two test certificates of under 1 KB take 79 s and 22.7 GiB, and 99 s and 27.1 GiB, in the kernel, where the native module with all six test checks builds in 1.1 s, and test certificates of 11 KB ran out of 96 GiB. The hand-built release rewrote the checker over tables of natural-number literals, without strings, arrays or hash maps, and a fresh build of it took 8,805.9 CPU-hours over 102,624 modules [31].

Most facts of our lighter results with hand-built versions do not convert either. Of the Hamming cube's three native facts [36], two convert, but its tree check timed out at 900 and at 3,600 s, so none of its four native theorems converts. Its hand-built version has 1,097 lines in 51 files. The single native fact of our Zhang–Zagier height bound ran out of memory both at 48 and at 96 GiB, and of 74 facts that we ran from eight more results with hand-built versions (on totally positive Mahler measures, maximum terms, homological percolation, Schur–Siegel–Smyth traces, Bohr radii, Romanoff's constant and the beat-the-average game), 3 converted, and most of the others ran out of memory or exceeded the recursion depth within minutes. Our six-body result [33] has a native headline theorem with 3,158 native facts. The random sample ran the 546 in the modules that its current repository builds and converted 56 (Section 4.6). We re-ran with the current library the 132 that had not run out of memory there and 46 of the 414 that had, and the same 56 convert, which completes 18 of the 97 theorems that use these facts but no headline theorem. None of the 37 resource failures that we re-ran at 3,600 s and 96 GiB converted.

Results without a hand-built version. Our paper on range-criterion counterexamples to the PPT^2 conjecture [34] has no kernel version. Of the 751 native facts of its core part, 540 convert, and so do 271 of the 679 that we ran of its 1,039 row facts, those of the cited theorems first. Most of the others ran out of 48 GiB, and 3 of the 20 that we re-ran at 96 GiB converted. As a result, 573 of the 998 native theorems that the paper cites, and every cited theorem in 86 of the 165 rows of its Table A.1, now have kernel-checked versions. Samples from four more results without kernel versions (thrackles, the Steiner ratio, projective planes of order 12 and depth-9 local optimality of 4×4 matrix multiplication schemes) converted none of their 22 facts: 15 ran out of 100 GiB within nine minutes, 5 exceeded the recursion depth, and 2 are blocked by `Float` primitives and `partial` loops. Our proof of $S(14) \geq 50$, published while this paper was being finished, rests on 43,100 native facts and has no kernel version. We did not run it: its native build alone took 1,274.5 CPU-hours.

Across these results, the conversion replaced a hand-built version where the native checking

functions were already within the kernel's reach, and there at about the hand-built version's cost, whether that version had evaluated the same functions (Theorem 1.1) or a checker rewritten for the kernel (3SUM/APSP). Facts that search large spaces or read large data still need checkers written for the kernel, and that is where most of the cost of the hand-built versions lies. A probe shows which facts these are: for $S(13) \geq 45$, the static pass and the probe runs took 2.2 CPU-hours.

9 Related work

Proof by reflection [5] underlies `decide`. Coq evaluates by compiled code inside the kernel's conversion check (`vm_compute` [20], `native_compute` [4]); proofs that use `native_compute` trust the OCaml compiler in a similar way, and Coq's kernel was given machine integers and persistent arrays for checking SAT certificates by reflection [37]. HOL4 derives computation theorems by rewriting (`computeLib` [3]). Isabelle's normalization by evaluation [1] runs compiled ML code and admits its results through an oracle, and Isabelle distinguishes evaluation by the simplifier (`code_simp`), which produces theorems, from evaluation by generated code (`eval`), which is trusted [22, 48]. Candle adds a verified in-logic computation primitive [7]. The four-colour theorem [19] was checked by Coq's kernel, while Flyspeck's enumeration of tame graphs [27] ran through Isabelle's code generator [23]. Primality certificates checked in a proof assistant go back to Caprotti and Oostdijk [8] and Grégoire, Théry and Werner [21]; Isabelle's Archive of Formal Proofs has Pratt certificates [16], and Mathlib's `norm_num` decides the primality of small numbers. General recursion in type theory has a long literature, including the Bove–Capretta method [6] and Equations [29]; our fuel lemma follows the standard pattern of unfolding a fixpoint a bounded number of times.

SAT certificates. Verified checking of SAT and SMT certificates (Cruz-Filipe et al. [17], Lammich [24], cake_lpr [30], SMTCoq [2]) is the model for Section 7. The tactic `bv_decide` runs its verified LRAT checker as compiled code [39], and so does LRAT-Catcher, which imports large certificates into Lean in chunks [41]. Mathlib's `lrat_proof` [40] has Lean's kernel check an LRAT proof through a proof term built step by step. Two recent prototypes evaluate a checker in Lean's kernel. LratK [42] is an RUP checker with a bitset assignment and a soundness proof. The prototype lean-dedep [43] translates `bv_decide`'s whole pipeline into a non-dependent program with a generated proof relating the two; its tactic `in_kernel bv_decide` proves $x \cdot 3 = x + x + x$ for 6-bit x in 48 s, and the 16-bit case does not finish within ten minutes. Our route differs in its bit-blaster written for the kernel, its clauses stored as bitmask pairs with unit witnesses, and its chunks in separate modules. On the same goal, with the same settings (one thread, a 40 GiB address-space limit) but on a different machine, it takes 1.1 s for the whole file at 6 bits, 2.3 s at 16 bits, 12 s at 64 bits and 651 s at 1,024 bits.

Lean. Core Lean 4 [18] offers `decide +kernel`, the fuel-based `WellFounded.Nat.fix`, and `cbv`/`decide_cbv`, which rewrite with equation lemmas and therefore also handle well-founded definitions, more slowly. A core issue (#5806 [15]) plans to rewrite recursive functions into `Nat.rec` with large fuel, and Mathlib has fuel rewrites for individual functions. The prototype lean-dedep [43] also proves, by the same induction from `WellFounded.fix_eq` as our fuel lemma, that a well-founded fixpoint whose relation is bounded by a natural-number measure equals `WellFounded.Nat.fix`; our lemma needs no measure, because after 2^{64} unfoldings it falls back to `WellFounded.fix`. Parth Shastri proposed a lazily generated accessibility proof inside `WellFounded.fix` itself (PR #9944 [14], not merged). For removing the tactic, ReplaceNativeDecide [44] is a prompt for AI agents that replaces individual uses with conventional proofs, and the `repair_proofs` tool of AXLE [38]

replaces `native_decide` by `decide +kernel` within a submitted file, keeping the original where the kernel fails. Mathlib's linter [46] flags every use of the tactic. The tool lean4checker [11] re-checks environments with Lean's own kernel; nanoda [26] and con-ron [45] are independent implementations, and Lean4Lean [9] reimplements the kernel in Lean, closely following the C++ one. SafeVerify [12], Comparator [10] and `lake check` are gates that refuse native axioms. We know of no other tool that converts the existing native facts of whole projects without editing them, or of an earlier measurement of what prevents conversion.

10 Discussion

For authors of verified software. Most facts in the study did not need compiled evaluation, so `decide +kernel` is the better first choice. The constructs in Table 2 made facts unconvertible with current core: data stored as `String` and parsed at check time, `HashMap` and other code over `USize` (the word-size case split in Section 4.8 now proves results that do not depend on the word size), transcendental `Float` functions, and code that goes through the metaprogramming layer. Decidability instances obtained by a cast, such as `simp <;> infer_instance`, block kernel evaluation; `decidable_of_iff` avoids the cast, since its proof is never evaluated. Native facts inside instances should carry certificates. The linter warns, when the tactic is written, about reachable blocking constants. Before a kernel version is written by hand, a probe with the converter is cheap and shows which facts need one (Section 8).

For Lean core. A definition of `List.toByteArray` that builds the array in one step, rather than by repeated `Array.push`, would make kernel evaluation of string literals linear instead of quadratic; the proved-equal replacement in Section 4.3 does exactly this outside core. Linear evaluation is not enough for very long literals: in our own proof of $S(13) \geq 45$, a 64 MiB literal unfolds to a list of characters as deep as the literal is long, and the kernel rejected its copy for recursion depth even with a 32 GiB stack (Section 8). Lemmas that characterize `String.replace` and the other pattern-generic string functions by lists would let proved replacements reach further, and iterator state types the kernel can see through would remove the largest class's blocking constant. Logical definitions for the hash primitives would unblock `HashMap`, and arrays that the kernel evaluates without going through lists would let core's own `bv_decide` pipeline run in the kernel. None of these changes touches the resource failures or Strata's `partial` parser, unsafe implementations and `partial_fixpoint`.

Limitations.

- The corpus is selected, and the random sample is drawn from a code-search frame without repositories of 400,000 KB or more and limited to Lean ≥4.27. Neither counts facts in modules that did not build or that name no native tactic, such as uses through a project's own macro. One drawn repository was skipped by a script error (Section 4.6).
- The time and memory failures of the corpus and the sample mostly reflect the study-time tool: for most of them the converter's own search for split points, not the kernel, ran out (Section 4.8).
- The runs used several builds of the tool (Appendix B) and up to 32 processes on one machine; a process that the machine killed for memory counts as the fact's own memory failure.
- The time and memory limits are fixed, and heavy facts may convert with more resources.

- On our own results we ran samples where a class of facts or a result was too large, and for the quantum error-correction paper and $S(13) \geq 45$ we count the facts of a sampled class that we did not run as unconverted (Section 8).
- The static pass is not proved to over-approximate; it missed the 13 casts, which a later diagnostic found.
- Independent checks cover the supplements of three packages and samples from the others, not every converted fact. Supplements cover only the modules that name a native tactic.
- The source rewriter does not rewrite uses inside macro definitions, and neither removal tool handles `bv_decide`; VeIR, for example, calls `bv_decide` through its own macro at 247 call sites.
- The evidence for robustness to PR #14892 is indirect: 51,747 conversions never evaluate a definition compiled through `WellFounded.fix`, the only kind whose evaluation the PR can affect, but only two functions were run on a nightly that contains it.

11 Conclusion

In 9 of 16 packages more than 90% of the native facts were re-proved by Lean's kernel without editing the project. All but 22 of the 51,329 re-run conversions outside definitions succeed with plain kernel evaluation, so `decide +kernel`, applied also to the facts inside definitions, would have served for most of them. On our own results, the converter replaced hand-built kernel versions at about their cost where the native checks were already within the kernel's reach, but not for the sorting-network bound or the other results whose checks search large spaces or read large data. The outputs we checked passed nanoda, and they passed con-ron where it could read them. Of the study's 1,057 failures, 825 trace to a few constructs, string processing at check time foremost, and 230 exceed our resource limits. Extensions made after the study, some of them against the random sample's failures, raise its pooled rate from 44.9% to 68.8% and its median from 77.8% to 100%. For projects that keep the tactic, the one-import switch and the source rewriter remove the native axioms that the kernel proves within a budget. On the same sample the rewriter's defaults removed 30% of them and kept every build within 1.7 times its original build time.

A Repositories

package	repository and commit	status
TrainVerify	`Tianshi-Xu/tainverify_lean` 9cfbfc78048e	studied
cvc5/logos	`cvc5/logos` 4cd3a10e302a	studied
Humanfia IChO 2026	`humanfia/hoa` 1c918ec9a96c	studied
formal-conjectures	`google-deepmind/formal-conjectures` c83a94d785d0	studied
longfellow-zk	`longfellow-zk/longfellow-zk` de5f9490c3d8	studied
BerryEsseen	`CoolRmal/BerryEsseen` bfb22608ad18	studied
asi-stack-book	`corbensorenson/asi-stack-book` 5ae6da1ef0b6	studied
Strata	`strata-org/Strata` 649cba18f6f7	studied
clean	`Verified-zkEVM/clean` 1a6c28426372	studied
rs-mca	`przchojecki/rs-mca` 93fba1be3f32	studied
sounio	`Sounio-lang/sounio` 3ef3dbaa6e94	studied
lido-srv3	`lfglabs-dev/lido-srv3-proof-closure` 2a182aa1d9fa	studied
cedar-spec	`cedar-policy/cedar-spec` c9a03407a96a	studied
aks	`girving/aks` c7fb62ed80a4	studied
sorting-network	`VarunVejalla/sorting-network` aad4ffcef720	studied
aeneas	`AeneasVerif/aeneas` 74aabd52f770	studied
FX	`GrigoryEvko/FX` e01b6e40bb6c	no native facts
evm-asm	`Verified-zkEVM/evm-asm` 7e65e4d02471	no native facts
brill-noether	`npflueger/brill-noether-graphs` 68c6891b0a99	no native facts
tengoku	`competemath/tengoku` 1487abe2fe69	no native facts
alphaproof-nexus	`google-deepmind/alphaproof-nexus-results` 0647711a7118	no native facts
ternary-goldbach	`gersh/ternary-goldbach-lean` 27df23af6a71	none in built modules
program_guide	`lusing/program_guide` b5eadb6b4c8a	no native facts
sparkle	`Verilean/sparkle` 1614cee83ac5	no native facts
ArkLib	`Verified-zkEVM/ArkLib` d023c5d31d76	no native facts
UFT	`iarnoldy/UFT` a90ce55c5773	did not build
tetrasemaphore	`IT-Help-San-Diego/tetrasemaphore` 833c2438fc3e	did not build
veil	`verse-lab/veil` 02791b2d1576	did not build
FloatLib	`lean-dojo/FloatLib` 1f5771b3e36a	did not build
llm-rust-lean-pipeline	`tobiaskrogh-ship-it/llm-rust-lean-pipeline` 89b8ef6d0a0c	did not build

B Reproducibility

The library is available as a Lake package at https://github.com/tejasvi/native-trust. The artifact (doi:10.5281/zenodo.23628416) holds the library, the add-ons, the tests with their expected outputs, the drivers and aggregators, the per-fact results of every run cited here and the logs of the facts that did not convert, the supplements and checker outputs, the sample frame and the script that draws the sample (seed 20261023), and an index that gives, for each number in this paper, the files and the command that reproduce it.

Source manifests are SHA-256 digests over the per-file digests of the library and Lake files. The library in the artifact is `6b378a8e`. The versions below are in the artifact under their manifests, with the runs that used them:

- `b0da4454`: the fuel route without the retry; used for the supplements, the sampled re-checks, the ablation and the random sample. Its seven library modules are in the artifact, but the root module and Lake files of that build were not kept, so this digest cannot be recomputed.
- `649b14e3`: adds the fuel route's retry; final runs of cvc5/logos, Strata and formal-conjectures. Version `727e381b` adds the restoration of the environment after a failed attempt, and `e839bc05` the one-import switch and the linter; re-runs of cvc5/logos, Strata and the 15 well-founded formal-conjectures facts with them gave the same results.

- `9011ea48`, `9f92c396`, `61dc5f97`, `72639b93`, `19a2dbb3` and `918ae06c`: the extensions in Section 4.8 (strings and large checks, casts, copies in types, strings as a retry and `KernelFirstLite`, the opt-in options, and the word-size case split with the rewriter's budget); the resource and BerryEsseen re-runs used `9011ea48`. Each was adopted only after the previously converted facts converted again (Section 4.8).

- `6b378a8e`: adds to `918ae06c` a check that the final theorem of the staged route uses only the standard axioms, as the other routes' theorems do; the test suite gave the same results with it on all 14 toolchains, and the five staged conversions of Section 4.8 were re-run with it. The runs on our own results (Section 8) also used it.

- `0d9bca14` and `c35df73b`: working builds that were not adopted. The first adds the changes for copies in types to `9011ea48` and ran 16 of the 17 facts blocked by them; the second applied the string replacements in the first attempt and ran the Strata check in which 32 facts timed out.

The other study packages were run with two earlier builds that differ from `b0da4454` in the well-founded route and the triage; the regression runs (Section 4.8) re-ran the converted facts of five of them and 2,018 of TrainVerify's with later builds and lost none.

References

[1] K. Aehlig, F. Haftmann, T. Nipkow. A compiled implementation of normalisation by evaluation. *J. Funct. Program.* 22(1), 2012.

[2] M. Armand, G. Faure, B. Grégoire, C. Keller, L. Théry, B. Werner. A modular integration of SAT/SMT solvers to Coq through proof witnesses. CPP 2011.

[3] B. Barras. Programming and computing in HOL. TPHOLs 2000.

[4] M. Boespflug, M. Dénès, B. Grégoire. Full reduction at full throttle. CPP 2011.

[5] S. Boutin. Using reflection to build efficient and certified decision procedures. TACS 1997.

[6] A. Bove, V. Capretta. Modelling general recursion in type theory. *Math. Struct. Comput. Sci.* 15(4), 2005.

[7] O. Abrahamsson, M. O. Myreen. Fast, verified computation for Candle. ITP 2023.

[8] O. Caprotti, M. Oostdijk. Formal and efficient primality proofs by use of computer algebra oracles. *J. Symb. Comput.* 32(1–2), 2001.

[9] M. Carneiro. Lean4Lean: verifying a typechecker for Lean, in Lean. arXiv:2403.14064v3, 2025.

[10] Comparator. `https://github.com/leanprover/comparator` (archived; part of Lean since 4.35 as `lake comparator`).

[11] lean4checker. `https://github.com/leanprover/lean4checker` (archived; distributed with Lean as `leanchecker` since 4.28).

[12] SafeVerify. `https://github.com/GasStationManager/SafeVerify`.

[13] J. Breitner. Lean 4 pull request #14892, "define well-founded recursion without large elimination of `Acc`". `https://github.com/leanprover/lean4/pull/14892`, 2026.

[14] P. Shastri. Lean 4 pull request #9944, "experiment: definitional well-founded recursion" (not merged). https://github.com/leanprover/lean4/pull/9944, 2025.

[15] J. Breitner. Lean 4 issue #5806, "Optimizing terms for kernel reduction (meta issue)". https://github.com/leanprover/lean4/issues/5806, 2024.

[16] S. Wimmer, L. Noschinski. Pratt's primality certificates. *Archive of Formal Proofs*, 2013.

[17] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt, M. Kaufmann, P. Schneider-Kamp. Efficient certified RAT verification. CADE 2017.

[18] L. de Moura, S. Ullrich. The Lean 4 theorem prover and programming language. CADE 2021.

[19] G. Gonthier. Formal proof—the four-color theorem. *Notices of the AMS* 55(11), 2008.

[20] B. Grégoire, X. Leroy. A compiled implementation of strong reduction. ICFP 2002.

[21] B. Grégoire, L. Théry, B. Werner. A computational approach to Pocklington certificates in type theory. FLOPS 2006.

[22] F. Haftmann, T. Nipkow. Code generation via higher-order rewrite systems. FLOPS 2010.

[23] T. Hales et al. A formal proof of the Kepler conjecture. *Forum Math. Pi* 5, 2017.

[24] P. Lammich. Efficient verified (UN)SAT certificate checking. CADE 2017.

[25] lean4export. https://github.com/leanprover/lean4export.

[26] nanoda. https://github.com/ammkrn/nanoda_lib; version 0.4.19 as distributed with the Lean 4.35 toolchain.

[27] T. Nipkow, G. Bauer, P. Schultz. Flyspeck I: tame graphs. IJCAR 2006.

[28] V. R. Pratt. Every prime has a succinct certificate. *SIAM J. Comput.* 4(3), 1975.

[29] M. Sozeau, C. Mangin. Equations reloaded: high-level dependently-typed functional programming and proving in Coq. *Proc. ACM Program. Lang.* 3(ICFP), 2019.

[30] Y. K. Tan, M. J. H. Heule, M. O. Myreen. cake_lpr: verified propagation redundancy checking in CakeML. TACAS 2021.

[31] T. S. Tomar. A kernel-checked Lean proof that sorting networks on 13 inputs need 45 comparators. Zenodo, 2026. doi:10.5281/zenodo.23552145. Native Lean version: doi:10.5281/zenodo.23504361.

[32] T. S. Tomar. A matching threshold of 2×10^{-3} for the rotated surface code under circuit-level noise. Zenodo, 2026. doi:10.5281/zenodo.23498520. Native Lean version: doi:10.5281/zenodo.23485205; kernel-only Lean version: doi:10.5281/zenodo.23498518.

[33] T. S. Tomar. Smale's sixth problem for six bodies in the plane: finiteness of central configurations for generic masses, with a proof formalized in Lean. Zenodo, 2026. doi:10.5281/zenodo.23472600. Supplement: doi:10.5281/zenodo.23472605.

[34] T. S. Tomar. On the minimal dimension of range-criterion counterexamples to the PPT^2 conjecture. Zenodo, 2026. doi:10.5281/zenodo.23651994. Lean checks: doi:10.5281/zenodo.23651991.

[35] T. S. Tomar. Which fine-grained lower bounds survive the refutation of 3SUM and APSP? Zenodo, 2026. doi:10.5281/zenodo.23609699. Supplement: doi:10.5281/zenodo.23609700.

[36] T. S. Tomar. The L^1 Poincaré constant of the Hamming cube. Zenodo, 2026. doi:10.5281/zenodo.23533350. Supplement: doi:10.5281/zenodo.23533351.

[37] M. Armand, B. Grégoire, A. Spiwack, L. Théry. Extending Coq with imperative features and its application to SAT verification. ITP 2010.

[38] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh, J. Limperg. AXLE: a cloud infrastructure for Lean 4 theorem proving utilities. arXiv:2606.26442, 2026.

[39] H. Böving, S. Bhat, L. Cicolini, A. Keizer, L. Frenot, A. Mohamed, L. Stefanesco, H. Khan, J. Clune, C. Barrett, T. Grosser. Interactive bitvector reasoning using verified bit-blasting. *Proc. ACM Program. Lang.* 9(OOPSLA2), 2025.

[40] M. Carneiro. `Mathlib.Tactic.Sat.FromLRAT` (the `lrat_proof` command). Mathlib, 2022.

[41] S. Szeider. Streaming LRAT certificates into Lean theorems. arXiv:2607.00815, 2026.

[42] LratK, an LRAT checker evaluated by Lean's kernel (file `CoveringLean/LratK.lean`). https://github.com/thiagopatzdorf/Matematica, 2026.

[43] lean-dedep (proof of concept). https://github.com/nomeata/lean-dedep, 2026.

[44] ReplaceNativeDecide. https://github.com/GasStationManager/ReplaceNativeDecide.

[45] con-ron. https://github.com/leanprover/con-ron.

[46] Mathlib pull requests #25297, "lint against using `native_decide`" (2025), and #43194 (`linter.style.native`, 2026). https://github.com/leanprover-community/mathlib4/pull/25297.

[47] J. Breitner. Lean 4 pull request #12887, "optimize string literal equality simprocs for kernel efficiency". https://github.com/leanprover/lean4/pull/12887, 2026.

[48] F. Haftmann. Code generation from Isabelle/HOL theories. Isabelle documentation (tutorial `codegen`).